41.1 TUI 架构

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~340k tokens，输出 ~6k tokens（本节）

OpenClaw 提供了一个全功能的终端用户界面（TUI，Terminal User Interface），让用户可以直接在终端中与 AI Agent 进行交互式对话。TUI 不仅仅是一个简单的命令行 REPL——它具备语法高亮、Markdown 渲染、工具调用可视化、多 Agent 切换、会话管理等完整功能。

> **衍生解释：TUI 与 CLI 的区别**
> CLI（Command Line Interface）是传统的命令行工具——用户输入命令、程序返回结果，交互是线性的。TUI（Terminal User Interface）则在终端中构建了类似 GUI 的界面，具有布局系统、交互组件、实时更新等特性。经典的 TUI 应用包括 `vim` 、 `htop` 、 `tmux` 等。TUI 通过 ANSI 转义码控制终端的光标位置、文字颜色和屏幕区域。

41.1.1 pi-tui 集成

OpenClaw 的 TUI 基于 `@mariozechner/pi-tui` 库构建。该库提供了一套终端 UI 组件系统，包括布局容器、文本渲染、编辑器、加载动画、列表选择等基础组件。

核心依赖

```
import {
  CombinedAutocompleteProvider,  // 组合自动补全
  Container,                     // 布局容器
  Loader,                        // 加载动画
  ProcessTerminal,               // 终端适配器
  Text,                          // 文本组件
  TUI,                           // TUI 根实例
} from "@mariozechner/pi-tui";
```

TUI 组件树

TUI 的界面由一棵组件树构成：

```
TUI (根)
└── root (Container)
    ├── header (Text)          ← 标题栏: URL / Agent / Session
    ├── chatLog (ChatLog)      ← 聊天记录区域
    ├── statusContainer        ← 状态行（加载动画 / 消息提示）
    ├── footer (Text)          ← 底部信息栏
    └── editor (CustomEditor) ← 输入编辑器
```

```
// src/tui/tui.ts（简化）

export async function runTui(opts: TuiOptions) {
  const tui = new TUI(new ProcessTerminal());
  const header = new Text("", 1, 0);
  const statusContainer = new Container();
  const footer = new Text("", 1, 0);
  const chatLog = new ChatLog();
  const editor = new CustomEditor(tui, editorTheme);

  const root = new Container();
  root.addChild(header);
  root.addChild(chatLog);
  root.addChild(statusContainer);
  root.addChild(footer);
  root.addChild(editor);

  tui.addChild(root);
  tui.setFocus(editor);  // 焦点默认在编辑器上
}
```

`ProcessTerminal` 是终端的底层适配器，封装了 `process.stdin` / `process.stdout` 的读写和终端尺寸检测。

自定义组件

除了 `pi-tui` 的内置组件，OpenClaw 还实现了以下自定义组件（位于 `src/tui/components/`）：

组件	文件	功能
`ChatLog`	`chat-log.ts`	聊天记录显示， 支持滚动和历史加载

组件	文件	功能
`CustomEditor`	`custom-editor.ts`	输入编辑器，支持多行和自动补全
`AssistantMessage`	`assistant-message.ts`	AI 回复的 Markdown 渲染
`UserMessage`	`user-message.ts`	用户消息的格式化显示
`ToolExecution`	`tool-execution.ts`	工具调用的可视化
`FilterableSelectList`	`filterable-select-list.ts`	可过滤的选择列表
`SearchableSelectList`	`searchable-select-list.ts`	可搜索的选择列表（模糊匹配）

41.1.2 TUI 适配层

`src/tui/` 目录的文件按职责清晰分层：

```
tui.ts                    — 主入口：组件树构建 + 状态管理
tui-types.ts              — 类型定义
├── gateway-chat.ts       — Gateway WebSocket 客户端适配
├── tui-event-handlers.ts — Gateway 事件处理
├── tui-command-handlers.ts — 斜杠命令处理
├── tui-session-actions.ts  — 会话操作（切换/重置/加载历史）
├── tui-stream-assembler.ts — 流式响应组装
├── tui-formatters.ts     — 文本格式化
├── tui-overlays.ts       — 覆盖层（Agent 选择器/会话选择器）
├── tui-local-shell.ts    — 本地 Shell 执行（!command）
├── tui-waiting.ts        — 等待状态消息
├── tui-status-summary.ts — Gateway 状态摘要
├── commands.ts           — 斜杠命令定义
└── theme/                — 主题和配色
```

Gateway 客户端

TUI 通过 `GatewayChatClient` 与 Gateway 通信：

```typescript
// src/tui/gateway-chat.ts（简化）

export class GatewayChatClient {
  private client: GatewayClient;

  constructor(opts: GatewayConnectionOptions) {
    this.client = new GatewayClient({
      url: resolveUrl(opts),
      token: opts.token,
      password: opts.password,
      clientName: GATEWAY_CLIENT_NAMES.TUI,
      mode: GATEWAY_CLIENT_MODES.TUI,
      onEvent: (evt) => { /* 转发事件 */ },
      onHelloOk: () => { /* 连接成功回调 */ },
      onClose: () => { /* 断开回调 */ },
    });
  }

  async sendMessage(opts: ChatSendOptions) {
    return this.client.request("chat.send", {
      sessionKey: opts.sessionKey,
      message: opts.message,
      thinking: opts.thinking,
      deliver: opts.deliver,
      idempotencyKey: opts.runId ?? randomUUID(),
    }, { expectFinal: true });
  }

  async listSessions(params) {
    return this.client.request("sessions.list", params);
  }

  async patchSession(params) {
    return this.client.request("sessions.patch", params);
  }
}
```

状态管理

TUI 的状态通过 `TuiStateAccess` 接口集中管理，使用 getter/setter 代理模式：

```typescript
export type TuiStateAccess = {
  agentDefaultId: string;       // 默认 Agent ID
  currentAgentId: string;       // 当前活跃 Agent
  currentSessionKey: string;    // 当前会话键
  currentSessionId: string | null;
  activeChatRunId: string | null; // 当前运行 ID
  historyLoaded: boolean;        // 历史是否已加载
  sessionInfo: SessionInfo;      // 会话信息（模型/token/thinking 等）
  isConnected: boolean;          // WebSocket 连接状态
  toolsExpanded: boolean;        // 工具输出是否展开
  showThinking: boolean;         // 是否显示思维过程
  connectionStatus: string;      // "connecting" / "connected" / "reconnecting"
  activityStatus: string;        // "idle" / "sending" / "waiting" / "streaming"
  // ...
};
```

状态变更会自动触发 UI 更新。例如， `activityStatus` 从 `"idle"` 变为 `"streaming"` 时，状态栏会切换为加载动画。

输入处理

编辑器的提交处理支持三种输入模式：

```
export function createEditorSubmitHandler(params) {
  return (text: string) => {
    const raw = text;
    const value = raw.trim();
    params.editor.setText("");

    if (!value) return;  // 忽略空输入

    // 模式 1: Bash 命令（以 ! 开头）
    if (raw.startsWith("!") && raw !== "!") {
      params.editor.addToHistory(raw);
      void params.handleBangLine(raw);
      return;
    }

    params.editor.addToHistory(value);

    // 模式 2: 斜杠命令（以 / 开头）
    if (value.startsWith("/")) {
      void params.handleCommand(value);
      return;
    }

    // 模式 3: 普通消息
    void params.sendMessage(value);
  };
}
```

三种输入模式：

前缀	模式	示例	行为
`!`	Bash	`!ls -la`	在本地 Shell 中执行命令
`/`	命令	`/think high`	执行斜杠命令
无	消息	帮我写一个函数	发送给 AI Agent

注意 `!` 模式使用**原始未裁剪的文本**（`raw`），确保前导空格不会误触发 Bash 模式。单独的 `!` 被视为普通消息。

41.1.3 终端渲染

主题系统

TUI 的配色方案定义在 `src/tui/theme/theme.ts` 中：

```
const palette = {
  text: "#E8E3D5",         // 主文本色
  dim: "#7B7F87",          // 暗色文本
  accent: "#F6C453",       // 强调色（金黄色）
  accentSoft: "#F2A65A",   // 柔和强调色
  border: "#3C414B",       // 边框色
  userBg: "#2B2F36",       // 用户消息背景
  userText: "#F3EEE0",     // 用户消息文本
  systemText: "#9BA3B2",   // 系统提示文本
  toolPendingBg: "#1F2A2F", // 工具执行中背景
  toolSuccessBg: "#1E2D23",  // 工具成功背景
  toolErrorBg: "#2F1F1F",    // 工具失败背景
  toolTitle: "#F6C453",    // 工具标题色
  quote: "#8CC8FF",        // 引用文本色（蓝色）
  code: "#F0C987",         // 行内代码色
  codeBlock: "#1E232A",    // 代码块背景
  link: "#7DD3A5",         // 链接色（绿色）
  error: "#F97066",        // 错误色（红色）
  success: "#7DD3A5",      // 成功色（绿色）
};
```

配色方案经过精心设计，在深色终端中提供舒适的阅读体验。代码高亮基于 `cli-highlight` 库，支持主要编程语言的语法着色。

终端基础工具

`src/terminal/` 提供了底层的终端操作工具：

文件	功能
`ansi.ts`	ANSI 转义码常量和工具函数
`palette.ts`	全局调色板（与 TUI 主题独立）
`links.ts`	终端超链接（OSC 8 协议）

文件	功能
`table.ts`	终端表格渲染
`stream-writer.ts`	流式写入器（支持节流和缓冲）
`progress-line.ts`	进度行（单行更新）
`note.ts`	提示框渲染
`restore.ts`	终端状态恢复（异常退出时还原）

流式响应组装

`TuiStreamAssembler` 负责将 Gateway 的增量事件组装为完整的显示文本：

```
export class TuiStreamAssembler {
  private runs = new Map<string, RunStreamState>();

  // 处理增量更新
  ingestDelta(
    runId: string,
    message: unknown,
    showThinking: boolean,
  ): string | null {
    const state = this.getOrCreateRun(runId);
    const previousDisplayText = state.displayText;

    // 分别提取思维过程和正文内容
    const thinkingText = extractThinkingFromMessage(message);
    const contentText = extractContentFromMessage(message);
    if (thinkingText) state.thinkingText = thinkingText;
    if (contentText) state.contentText = contentText;

    // 合成显示文本
    state.displayText = composeThinkingAndContent({
      thinkingText: state.thinkingText,
      contentText: state.contentText,
      showThinking,
    });

    // 只在文本实际变化时返回
    if (state.displayText === previousDisplayText) return null;
    return state.displayText;
  }

  // 终结：清理运行状态
  finalize(runId: string, message: unknown, showThinking: boolean) {
    // ... 最终组装 + 清理
    this.runs.delete(runId);
    return finalText;
  }
}
```

每个运行（ `runId` ）维护独立的状态： `thinkingText` （思维过程）和 `contentText` （正文内容）分别追踪，按需合成显示文本。 `showThinking` 参数控制是否将思维过程（如 Claude 的 `<thinking>` 标签内容）显示给用户。

状态行与等待动画

当 Agent 正在处理时，状态行显示动态的等待消息：

```
const busyStates = new Set([
  "sending",    // 正在发送消息
  "waiting",    // 等待 Agent 响应
  "streaming",  // 接收流式输出
  "running",    // Agent 执行工具
]);

// 每秒更新状态行
const startStatusTimer = () => {
  statusTimer = setInterval(() => {
    if (!busyStates.has(activityStatus)) return;
    updateBusyStatusMessage();
  }, 1000);
};

// 等待消息示例: "thinking... • 12s | connected"
```

`tui-waiting.ts` 提供了一组随机等待短语（如"思考中..."、"正在推理..."），增加界面的趣味性。

41.1.4 斜杠命令系统

TUI 提供了丰富的斜杠命令，涵盖 Agent 管理、会话控制、模型切换等：

```
export function getSlashCommands(options): SlashCommand[] {
  return [
    { name: "help", description: "Show slash command help" },
    { name: "status", description: "Show gateway status summary" },
    { name: "agent", description: "Switch agent (or open picker)" },
    { name: "session", description: "Switch session" },
    { name: "model", description: "Set model (or open picker)" },
    {
      name: "think",
      description: "Set thinking level",
      getArgumentCompletions: (prefix) =>
        thinkLevels.filter(v => v.startsWith(prefix))
          .map(value => ({ value, label: value })),
    },
    { name: "verbose", description: "Set verbose on/off" },
    { name: "reasoning", description: "Set reasoning on/off" },
    { name: "elevated", description: "Toggle elevated mode" },
    { name: "activation", description: "Set group activation mode" },
    { name: "usage", description: "Toggle usage footer" },
    { name: "queue", description: "Adjust queue mode" },
    { name: "bash", description: "Run a host command" },
    { name: "compact", description: "Compact session history" },
    { name: "new", description: "Start new session" },
    { name: "reset", description: "Reset session" },
    { name: "stop", description: "Stop current run" },
    // ...
  ];
}
```

命令支持**参数自动补全**——例如 `/think` 命令会补全 `off`、`minimal`、`low`、`medium`、`high`、`xhigh` 等级别。命令别名机制（`COMMAND_ALIASES`）支持简写，如 `/elev` → `/elevated`。

覆盖层选择器

复杂的选择操作（如 Agent 切换、会话选择、模型选择）通过**覆盖层**（overlay）实现：

```
// src/tui/tui-overlays.ts
// Agent 选择器 → 模糊搜索列表
// 会话选择器 → 可过滤列表 + 会话元数据预览
// 模型选择器 → 可搜索列表 + 模型能力信息
```

覆盖层使用 `SearchableSelectList` 组件，支持键盘导航和模糊搜索（`fuzzy-filter.ts`），提供类似 VS Code 命令面板的交互体验。

41.1.5 本地 Shell

TUI 的 `!` 前缀支持直接在终端中执行本地 Shell 命令：

```
// src/tui/tui-local-shell.ts

export function createLocalShellRunner(params: {
  chatLog: ChatLog;
  editor: CustomEditor;
}) {
  return async (bangLine: string) => {
    const command = bangLine.slice(1); // 去掉 ! 前缀
    // 在 chatLog 中显示命令
    // 通过子进程执行
    // 实时流式显示输出
    // 错误处理
  };
}
```

这使得用户无需离开 TUI 界面就能执行文件操作、运行脚本等任务——在 AI 辅助编程的工作流中非常实用。

本节小结

1. **pi-tui 框架**：TUI 基于 `@mariozechner/pi-tui` 构建，使用组件树模型（header → chatLog → status → footer → editor）。
2. **Gateway 通信**：`GatewayChatClient` 封装了 WebSocket 客户端，提供 `sendMessage` / `listSessions` / `patchSession` 等高层 API。
3. **三种输入模式**：`!` 前缀执行本地 Shell、`/` 前缀触发斜杠命令、无前缀发送消息给 Agent。
4. **流式响应组装**：`TuiStreamAssembler` 分别追踪思维过程和正文内容，按需合成显示文本，支持增量更新。
5. **主题系统**：精心设计的 20+ 色调色板，支持语法高亮（`cli-highlight`），在深色终端中提供舒适的阅读体验。
6. **斜杠命令**：20+ 个命令覆盖 Agent/会话/模型管理，支持参数自动补全和命令别名。

7. **覆盖层选择器**：模糊搜索列表用于 Agent/会话/模型选择，提供类似 VS Code 命令面板的交互体验。
8. **终端工具层**： `src/terminal/` 提供 ANSI 控制、表格渲染、流式写入、超链接等底层能力。